Summer Semester 2025 - Guided Research Report

# Enabling GPU Passthrough for MicroVMs

Muhammed Orhun Gale
Technische Universität München

October 15, 2025

## Abstract

MicroVMs power serverless platforms by booting quickly, using little memory, and preserving strong isolation. At the same time, many workloads now expect access to GPUs, which are discovered and driven through PCIe. This work asks how far we can go toward "bare-metal" GPU use while keeping Firecracker's minimal design intact. We add an opt-in path for full-device passthrough that exposes just enough PCI to the guest and relies on VFIO with IOMMU protection, allowing stock NVIDIA drivers to run inside the microVM. We evaluate the approach on a modern GPU host, focusing on the two things that matter most for serverless scheduling: time from VMM start to a GPU-ready guest and steady-state footprints during idle and a small CUDA workload. The results show a clear trade-off: passthrough introduces extra seconds in the cold path and reduces host memory headroom due to pinned DMA mappings, while the steady compute path remains close to host behavior and the guest's idle footprint stays essentially unchanged. We discuss the implications for scale-to-zero and consolidation, and we provide artifacts to reproduce our measurements.

**Keywords:** microVM, GPU Passthrough

## 1 Introduction

Public clouds run many tenants on the same hardware and must grow and shrink capacity on short notice. To do this at scale, providers rely on serverless platforms that hide machine management, meter execution at fine granularity, and place short-lived tasks densely on shared hosts [5, 14]. The economics only work if the isolation boundary adds little overhead in the control path and in memory while still treating each workload as untrusted [3]. Measurements from production traces confirm that function

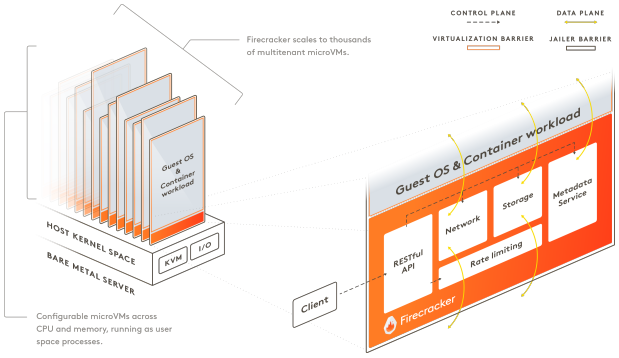


**Figure 1:** Firecracker's control/data-plane layout and microVM scaling [2].

lifetimes are short and bursty, which puts pressure on cold-start latency and per-instance footprint [15]. In parallel, AI workloads have moved from experiments to production services. Inference dominates sustained usage in many fleets and drives energy and cost, which means platforms that schedule short-lived tasks must expose accelerators without undermining multi-tenant safety or agility [9, 17]. Recent surveys point to continued growth in accelerator-backed deployments across cloud and HPC settings, reinforcing the need to integrate GPUs into isolation-centric runtime models [7, 17].

MicroVMs were introduced as a practical answer to these constraints. A microVM is a KVM-based virtual machine with a minimal device model and a small control plane. Firecracker is the canonical implementation and has been deployed in large serverless and container hosting systems [3]. The design removes legacy device emulation, keeps a narrow set of paravirtual devices, runs one VMM process per guest, and shortens the boot path [3]. Figure 1 sketches this layout and the separation between control and data planes. The result is sub-second boot and a few megabytes of overhead while preserving hypervisor isolation, which enables high packing density on shared hosts [4]. Earlier work on lightweight vir-

tualization reached the same conclusion on different stacks, namely that VM isolation can be kept while cutting device surface and control complexity to reach near-container agility [11]. These properties define the baseline to be kept intact when adding accelerator access [3, 4].

The challenge addressed here follows directly from that design point. Firecracker omits PCI on purpose to keep the trusted computing base small and to reduce attack surface [3]. Commodity GPUs, however, are discovered and programmed as PCIe devices. In virtualized settings, direct device assignment relies on VFIO in user space together with an IOMMU to constrain DMA and route interrupts safely to a guest [8]. VFIO exposes devices through file descriptors under IOMMU protection, and it treats IOMMU groups as the unit of isolation, which prevents a device from issuing DMA outside a set of mapped guest regions [8, 18]. Figure 2 summarizes the VFIO/IOMMU path and why remapping is required to fence DMA. Without some form of PCI exposure in the guest, Firecracker cannot assign a real GPU, yet adding that exposure can increase the device surface and control-path complexity. The work therefore asks how much PCI and VFIO support is necessary to operate a GPU in a microVM and what the cost of doing so is in practice [10].

There are several ways to provide GPUs to guests, each with distinct implications for performance, consolidation, isolation, and control complexity. Full device passthrough gives a VM exclusive access to a physical GPU through VFIO and an IOMMU. It preserves vendor driver behavior and has been shown to approach native performance when the platform and device are configured correctly, which is why it is the common baseline for direct assignment [6, 16]. SR-IOV, when implemented by the GPU, exposes multiple virtual functions that can be assigned to different VMs with hardware isolation and IOMMU protection, improving consolidation at the cost of device and scheduler complexity in the host and guest stack [12, 7]. Mediated devices create virtual GPUs backed by a host driver that multiplexes the physical GPU in time or space; this increases sharing opportunities and can fit a broader range of hardware, but the guest experience depends on vendor software and may differ from bare metal for some features such as low-level performance counters or specific reset paths [13, 12]. A platform may combine these approaches as hardware and workload permit, but the baseline capability is to run one real GPU inside a VM safely with minimal integration effort and predictable driver semantics [6].

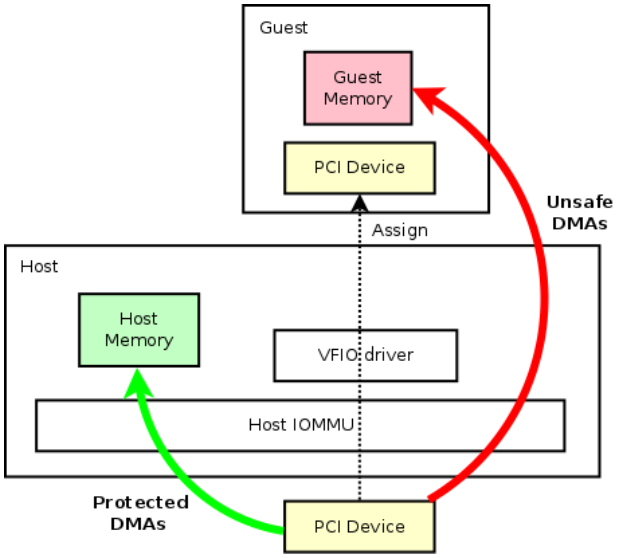


**Figure 2:** VFIO device assignment with IOMMU fencing of DMA [1].

This report focuses on that baseline. Full device passthrough for a GPU inside a Firecracker microVM is enabled with an opt-in PCI path and a VFIO device model. The decision is motivated by three simple requirements that mirror how accelerators are used in production. First, the guest should run unmodified vendor drivers so applications and frameworks see the same interfaces as on bare metal [12]. Second, the isolation boundary must be enforced by the IOMMU, with explicit DMA map and unmap operations and correct interrupt delivery, so the device cannot access memory outside the assigned guest [8]. Third, the VMM must preserve the short boot time and small memory overhead that justify microVMs in serverless scheduling [3, 4]. These requirements point to VFIO and PCIe as the right primitives, provided the PCI surface in the VMM remains narrow and disabled unless requested, which matches Firecracker's principle of keeping optional features outside the default path [10, 3].

Concretely, the guest must be able to enumerate the device and program its BARs and capabilities. On x86 this can be done through IO-port based configuration cycles and, optionally, through an ECAM region described via ACPI MCFG so that the guest can use memory-mapped configuration access where available [3]. Interrupts should use MSI or MSI-X, injected from the VMM through irqfd, to avoid reliance on legacy INTx routing while keeping guest drivers unchanged; this aligns with how modern guest drivers expect to signal work and reduces routing complexity [6, 3]. Every guest memory region that the device can access must be mapped into the VFIO container,

which pins those pages and limits overcommit while the device is attached; map and unmap operations must follow the guest memory lifecycle to avoid stale DMA windows [8, 18]. These choices keep the guest experience identical to a normal VM with a passed-through GPU and place the complexity at the VMM boundary where it can be audited and minimized without altering the default Firecracker configuration [3].

The implementation boundary is narrow by design. Device hotplug, live migration, or snapshot and restore of GPU state are not attempted, because these paths require capturing device and driver state that is not standardized across vendors or models [12]. SR-IOV or mediated sharing are not introduced, since those mechanisms change consolidation and scheduling assumptions and require extra host coordination to arbitrate resources across guests [7]. Cold-plug passthrough is sufficient to study the integration cost and to establish whether a microVM can host a GPU without losing the operational properties that matter for serverless; it isolates the problem to the minimum viable device model and avoids conflating sharing policies with the baseline ability to run a physical accelerator inside a microVM [10, 6]. This scoping also mirrors how minimal VMMs add first-class devices, where the initial target is a stable cold-boot path with clear teardown semantics [3].

With that scope defined, a specific research question follows: how well does full GPU passthrough integrate with a microVM designed for minimalism and what concrete costs does it add relative to a standard microVM and to host execution? The evaluation must measure the time from VMM start to a state where the GPU is usable inside the guest, because serverless traces show that cold-start dominates user experience for short invocations [15]. It must also track the steady-state resource footprint of the VMM and the guest with and without the GPU attached, since Firecracker's value proposition includes high density through small per-instance overhead [3, 4]. Finally, a small CUDA microbenchmark should separate first-use effects such as context creation, allocation, and host–device transfers from kernel execution, so that any overheads attributable to the VMM's control path can be distinguished from the GPU's steady-state compute path [6]. These metrics align with how microVMs are evaluated and with how accelerator-backed services behave in production, and they allow comparison against Firecracker's published baselines for boot and footprint [4].

The remainder shows that a minimal, opt-in PCI and VFIO path in Firecracker is sufficient to operate a modern GPU inside a guest with standard drivers, without disturbing the default configuration. The VMM mechanisms needed for configuration space access, BAR mapping, interrupt delivery, and DMA containment are documented, and the impact on boot-to-GPU-ready time and on steady-state resource use is quantified in execution windows that matter for serverless [8, 3]. The analysis relates the observed overheads to the initialization path, including enumeration, driver bring-up, and context creation, and it outlines the operational limits that follow for multi-tenant use at scale, such as pinned memory during DMA and the absence of snapshot support for assigned devices [18, 7].

# 2 Literature Review

## 2.1 PCI/PCIe in virtual machines

PCIe is the discovery and configuration backbone for discrete devices, including GPUs, in bare-metal and virtualized systems. Guests enumerate functions, read capability lists, and size BARs through configuration space, then map register and memory windows into their address space [3]. On x86, configuration cycles can flow through IO-port routed CF8/CFC or through ECAM regions described by ACPI MCFG, which lets the guest access extended configuration space directly and enables clean capability programming such as MSI and MSI-X [3]. In a conventional VMM this layout is standard, so a passed-through device appears to the guest as a normal PCIe endpoint whose BARs back MMIO mappings and whose interrupts are message signaled, not legacy INTx [8]. Firecracker departs from this norm by design. It strips legacy emulation and excludes PCI entirely to keep the device surface and the trusted computing base small, which shortens the boot path and reduces memory overhead in multi-tenant settings [3, 4]. That choice is compatible with virtio over MMIO for disk and network, yet it blocks the default way to expose discrete accelerators that only speak PCIe [3]. Public maintainer discussions acknowledge the tension and explore staged, opt-in PCI support that does not change the default microVM behavior [10].

## 2.2 VFIO and IOMMU for direct assignment

Linux provides VFIO as the user space interface for safe device assignment. VFIO binds a PCI

function to a userspace driver, enforces isolation through the IOMMU, and exposes ioctls for DMA map and unmap so a VMM can constrain a device to the guest physical ranges it owns [8]. Isolation is organized around IOMMU groups, which reflect hardware boundaries in the PCIe topology, so a function can only be assigned if its group contains no unrelated devices, a policy that prevents stray DMA from escaping the sandbox [8]. The classic analysis of direct assignment explains why this indirection matters: without remapping, devices can reach arbitrary host memory; with VT-d or AMD-Vi in front of the bus, DMA targets are translated and checked, which gives a VM near bare-metal semantics without surrendering memory safety [18]. In practice the VMM maps guest RAM into the VFIO container at region granularity, wires eventfds to irqfd, and lets the guest program MSI or MSI-X as usual, so interrupt delivery becomes an edge-triggered event injection rather than level arbitration [8]. Enterprise operator guides mirror the same steps and highlight operational side effects that matter for scheduling. Pages pinned for device DMA restrict memory overcommit while the device is attached, and assignment must follow group boundaries, which fixes how GPUs can be carved up at the PCIe level [13]. These constraints are not quirks of a particular stack, they are the standard cost of secure assignment on commodity x86 platforms [8, 13].

### 2.3 GPU passthrough in practice and constraints for microVMs

Empirical studies set expectations for full-device passthrough. A comprehensive survey reports that assigning a whole GPU to a VM yields performance close to native across common kernels when the IOMMU and MSI paths are configured correctly, and that most overhead concentrates in initialization rather than in the data path [6]. Earlier experiments on KVM and Xen measured penalties in the low single digits, with steady-state throughput dominated by the device and driver, not the VMM [16, 6]. The mechanism explains the result. Once BARs are programmed and DMA windows are established, the guest uses the vendor driver unchanged, kernels launch as on bare metal, and the VMM mostly handles irqfd notifications and accounting [6, 8]. These data points are the baseline for any minimal VMM that aims to expose a real GPU safely.

The design space does include sharing mechanisms, but they carry different trade-offs. With SR-IOV, a device advertises multiple virtual functions that the hypervisor can assign to different guests, each enforced by the IOMMU, which improves consolidation at the cost of device support and a more involved scheduling story inside the GPU stack [12, 7]. With mediated devices, a host driver time-slices or partitions the GPU and presents virtual GPUs whose feature set depends on the vendor stack; this increases sharing opportunities and works across a wider range of hardware, yet it moves more trust into host software and may diverge from bare-metal semantics for some features [13, 12]. For a microVM platform whose goal is to keep the guest environment standard and the control plane small, full-device passthrough is the capability to establish first, because it reuses stock drivers and avoids paravirtual contracts on the accelerator path [3, 6].

Firecracker adds two product constraints on top of this literature. First, any PCI surface must be off by default so the common case keeps the published boot and footprint targets that enable high packing density for short-lived tasks [4, 3]. Second, the implementation must not undermine isolation guarantees that serverless platforms rely on, which means IOMMU on the host, explicit DMA mapping in the VMM, and MSI or MSI-X delivery rather than legacy INTx, with the guest enumerating the device through IO-port configuration cycles or ECAM exposed via ACPI MCFG [8, 3]. These requirements are consistent with the usage trend on the workload side. Modern services rely on accelerators for training and, even more, for sustained inference, so a microVM-based platform must make GPUs available without giving up the control-path agility that production traces show is necessary for bursty invocations [17, 15]. The literature therefore frames a concrete target for this work: add just enough PCI and VFIO to let a stock guest driver operate a GPU with IOMMU protection, keep the path opt-in, and measure the cost relative to a normal Firecracker microVM [10, 4, 6].

## 3 Methodology

This section describes how full-device GPU passthrough was enabled and evaluated in a Firecracker microVM. The goal is to keep the control plane small, preserve the default boot path for non-GPU guests, and let a stock NVIDIA driver bind to a real device inside the guest. The platform, software stack, VMM changes, the VFIO and IOMMU path, interrupt delivery, configuration surface, boot and

readiness signals, metrics, and controls are detailed.

## 3.1 Goals and scope

The target is a minimal, cold-plug GPU passthrough path that keeps Firecracker's default configuration unchanged for non-GPU workloads. One physical GPU is attached to a microVM, a conventional PCIe view is exposed to the guest, and VFIO and IOMMU are relied upon for isolation. Device hotplug, live migration, or snapshot and restore while a device is attached are not attempted. Sharing mechanisms such as SR-IOV and mediated devices are out of scope. The evaluation quantifies costs that matter to serverless scheduling: time to a GPU-ready state after VMM start and steady-state footprints in idle and active windows.

## 3.2 System and Environment Settings

### 3.2.1 Hardware platform

Experiments run on a dual-socket AMD EPYC 9374F host with two NVIDIA H100 NVL 94,GB boards. One GPU is reserved for passthrough, the other is idle on the host. The host display is a separate low-end adapter to avoid binding issues on the primary console. IOMMU is enabled in firmware and in the kernel command line. NUMA placement is fixed so the microVM and its assigned GPU share a proximity domain. CPU isolation is not used for data path measurements; pinning is applied only where noted in controls.

### 3.2.2 Host software and drivers

The host runs Ubuntu 24.04.2 with a 6.8.0-79 kernel, NVIDIA driver 570.172.08, and a userspace stack that includes `vfio-pci`, `iommufd`, and recent `liburing`. Firecracker is v1.12.0 with PCI and VFIO additions compiled in but disabled by default. Devices selected for assignment are unbound from native drivers, attached to `vfio-pci`, and validated to sit in their own IOMMU groups. The host does not overcommit memory for the VM while the device is active, since DMA mappings pin guest pages.

### 3.2.3 Guest image and kernel

The guest image is a minimal Ubuntu 24.04 rootfs with a recent generic kernel and the standard NVIDIA userspace. The kernel includes PCI core support, ECAM, MSI/MSI-X, VFIO-related UAPI, and the necessary filesystems for a static userspace. The vendor CUDA runtime matching the host driver is installed, and any persistence daemons that could mask first-use costs are disabled. SSH is not required; all control is through the serial console and a small init script.

## 3.3 Firecracker Modifications

At a high level, the configuration spans user space, kernel space, and hardware. Firecracker runs in user space, interacts with `/dev/kvm` to drive vCPUs, and manages the microVM (vCPU, RAM, virtio, and the passed-through GPU). In the kernel, KVM provides virtualization, while VFIO (core and `vfio_pci`) together with the IOMMU subsystem (via `iommufd`) mediate safe device assignment. The physical GPU on the host is bound to `vfio-pci`. Figure 3 summarizes these roles and bindings that the subsequent subsections configure in detail.

### 3.3.1 PCI host, config space, and BARs

Firecracker's default device model remains intact. The PCI path is opt-in behind a configuration switch. When enabled, the VMM instantiates a single PCI host bridge on bus 0, reserves deterministic MMIO windows for BARs, and exposes an ACPI MCFG table so the guest can use ECAM for configuration space access. Legacy INTx is not routed. The allocator carves non-overlapping apertures for 64-bit and 32-bit BARs, aligned to device requirements, and programs them before the guest probes drivers. The ACPI tables are built at VM creation, loaded via the standard RSDP path, and validated by the guest during early boot. These changes mirror conventional VMM practice but keep the surface small and predictable [3].

### 3.3.2 VFIO path and IOMMU mappings

Device assignment uses VFIO with `iommufd`. At VM build time, the VMM opens the VFIO container, verifies that the target GPU's IOMMU group is isolated, attaches the group, and installs an address space for the guest. Large guest memory regions are mapped into the container with page-granular DMA windows. Guest RAM that is never shared with the device is left unmapped. Map and unmap operations track the guest memory lifecycle and the VM teardown path. The VFIO file descriptors are integrated into Firecracker's event loop so DMA windows and interrupts are handled without blocking the VMM thread

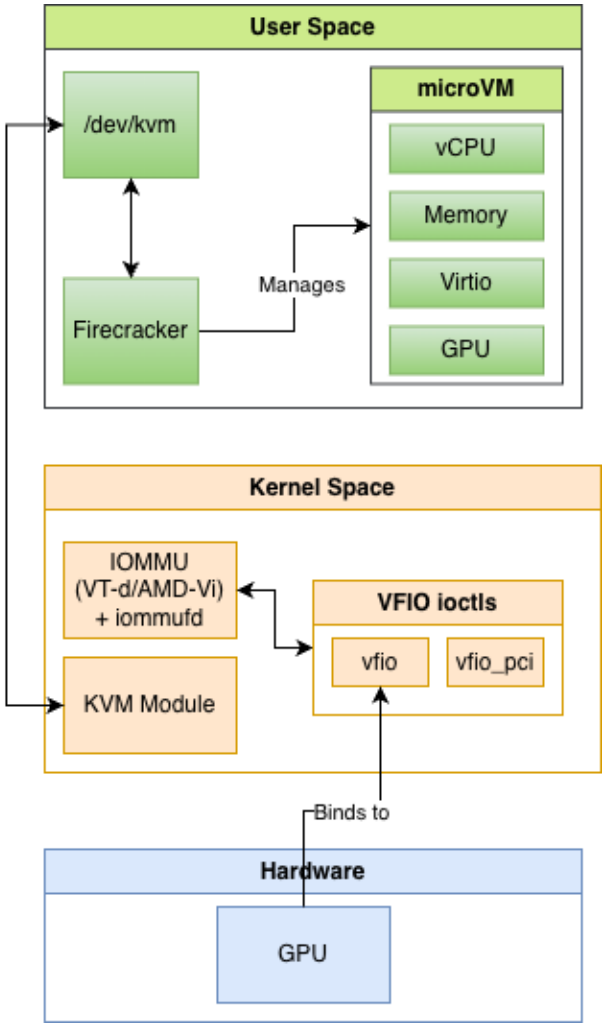


**Figure 3:** Stack overview across user space, kernel space, and hardware. Firecracker manages the microVM and interacts with `/dev/kvm`; in the kernel, KVM provides CPU virtualization and VFIO (`vfio`, `vfio_pci`) with the IOMMU subsystem (`iommufd`) enforce safe device assignment; the physical GPU is bound to `vfio-pci`.

that drives the guest CPUs. The isolation contract is the standard one: the IOMMU prevents the device from reaching host memory outside the mapped guest ranges [8].

### 3.3.3 Interrupt delivery

The guest enables MSI or MSI-X as negotiated by the vendor driver. For each allocated vector, the VMM programs the VFIO route and binds an `irqfd` to an `eventfd`. Interrupt injection is edge-triggered and does not depend on legacy line re-arbitration. The VMM's handler acknowledges host events and signals the corresponding virtual interrupt to the guest. This path mirrors the virtio MSI route already present in Firecracker, and it avoids introducing a second delivery mode in the PCI stack.

Figure 4 consolidates the control and data paths. Steps (1)–(7) mark, respectively: guest configuration-space access; BAR assignment into fixed MMIO32/64 apertures; VFIO operations to program BARs, map/unmap DMA, and arm IRQs; DMA translations through `iommufd`/IOMMU to guest RAM; device PCIe traffic; MSI/MSI-X remapping to a host IRQ and signaling via `eventfd`/`irqfd`; and injection of the resulting virtual interrupt to the guest LAPIC. This schematic corresponds to the mechanisms described in "PCI host, config space, and BARs," "VFIO path and IOMMU mappings," and "Interrupt delivery."

### 3.3.4 Configuration interface

The configuration remains a single JSON document. The PCI subsystem is off unless explicitly enabled. Passthrough devices are listed by sysfs path to make selection unambiguous:

```
{
  "pci": {
    "enabled": true,
    "vfio_devices": [
      { "path": "/sys/bus/pci/devices/0000:21:00.0/" }
    ]
  },
}
```

Other fields follow the normal microVM configuration: two vCPUs, 4,GiB RAM, `tap` network, kernel image, command line, and an ext4 rootfs. This keeps the GPU path opt-in and avoids changing defaults for non-GPU guests [3].

### 3.3.5 Boot sequence and readiness signal

On the host, the instant just before invoking Firecracker's `CreateVm` API is recorded as the start time. The guest's `init` brings up basic services, loads the NVIDIA driver, and runs a small probe program that creates a CUDA context, performs a fixed device memory allocation, and launches a trivial kernel. On success, the program prints a single `GPU_READY` token with a guest timestamp to the serial console and exits. The host reads this line from the serial pipe, the receipt time is recorded, and the boot-to-GPU-ready delta is computed from host monotonic clocks. The guest timestamp is logged only for sanity checks. This signal tests enumeration, BAR programming,

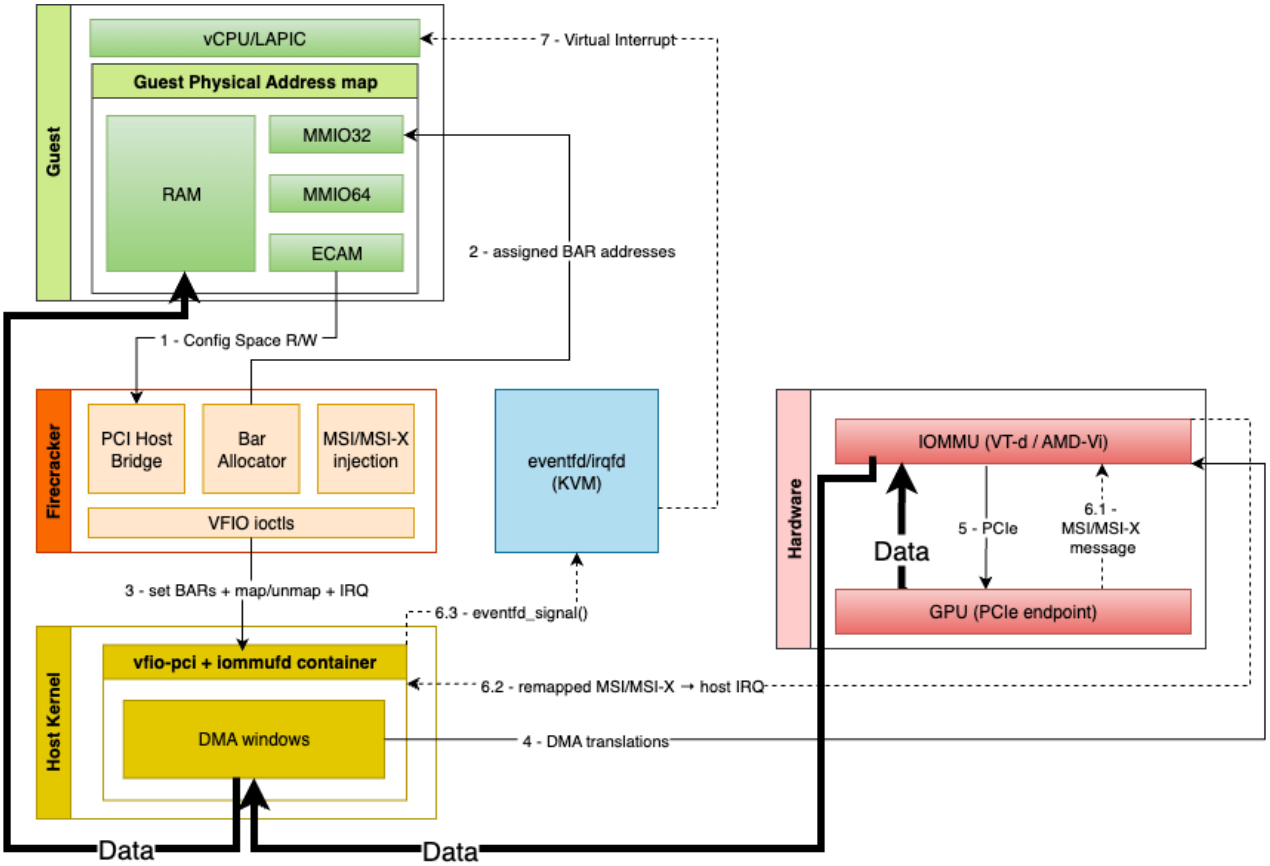


**Figure 4:** Minimal PCI/VFIO surface used by the prototype. The guest programs BARs into fixed MMIO apertures and enables MSI/MSI-X; Firecracker issues VFIO ioctls to map/unmap DMA windows in an `iommufd` container and arms interrupts. DMA requests are translated by the IOMMU (VT-d/AMD-Vi), and device MSI/MSI-X messages are remapped to a host IRQ and signaled via `eventfd/irqfd` for injection as a virtual interrupt.

MSI/MSI-X delivery, driver bring-up, and first-use effects such as context creation.

## 3.4 Experimental Setting

### 3.4.1 Microbenchmark and measurement windows

The microbenchmark is a fixed-size VectorAdd with explicit timing around five phases: context creation, host allocation, host-to-device transfer, kernel execution, and device-to-host transfer. A fixed number of warmup iterations is run, followed by measured repetitions that fit within a short execution window. This isolates first-use costs from steady-state compute. In addition to the microbenchmark, two steady-state windows are recorded: an idle period with no GPU attached and an idle period with the GPU attached and the driver loaded. These windows capture the incremental CPU time and RSS attributable to the PCI and VFIO path when the device is present but idle.

### 3.4.2 System resource sampling

Host and VMM resource counters are sampled at 1,Hz. On the host, the Firecracker process RSS, `cputime`, and file descriptor counts are tracked. For the guest, virtio block and net queues are tracked to confirm no background I/O patterns skew the results. Where available, Firecracker metrics for block device operations and API activity are read to guard against silent retries in the control plane. GPU board power and clocks are not part of the core metrics, but `nvidia-smi` telemetry is logged to confirm the device is quiescent during idle windows and that kernel clocks are consistent across runs. Sampling infrastructure runs out-of-band and does not reuse the serial console so coupling is avoided.

#### 3.4.3 Experiment design and baselines

Each configuration is run in batches of twenty trials per metric, where medians across 20 trials are reported for each metric. The baseline is a standard Firecracker microVM with no PCI support compiled in and no GPU assigned. The experimental variant enables PCI and attaches a single GPU. For sanity, the microbenchmark is also run on bare metal on the same GPU under the same driver and CUDA runtime. All runs use the same kernel image and rootfs where possible and the same guest command line. The page cache is flushed between trials to keep file system effects from masking boot path variance, and the VM is recreated per trial to avoid cross-trial state.

#### 3.4.4 Controls

The GPU is bound to `vfio-pci` only for the duration of the experiments and returned to the vendor driver afterward. It is verified that the GPU sits alone in its IOMMU group and that no audio or USB functions share the group. PCIe ASPM and power management settings are left at kernel defaults. CPU frequency scaling remains enabled, but current governors are recorded. The Firecracker VMM thread and the vCPU threads are pinned only in runs that explicitly test pinning; otherwise the scheduler assigns them normally. The guest's entropy source is pre-seeded so driver initialization does not stall. Network interfaces are present but unused. No other workload runs on the passthrough GPU during trials.

#### 3.4.5 Correctness checks

Before measuring, validation is performed to ensure the guest can enumerate the GPU, the vendor driver binds, and MSI or MSI-X vectors are active. A smoke test is run that allocates device memory, performs a transfer in both directions, and launches a simple kernel. AER errors on the host and IOMMU faults are checked. Any trial that reports PCIe errors, GPU Xid faults, or VFIO container failures is discarded and rerun. Serial logs are scanned for kernel oops and driver warnings. The teardown path is checked to ensure all DMA mappings are released and the device returns to the host driver cleanly.

#### 3.4.6 Failure handling

If the guest reports driver probe failures, dmesg from the serial console and the host VFIO logs are captured. If MSI allocation fails, vector counts are downgraded and retried. If IOMMU group checks fail, the device is not assigned, and the run is skipped. Any trial that drops into legacy INTx is marked invalid. If BAR apertures collide with guest RAM or other MMIO ranges, gaps are increased and the VM is rebuilt.

#### 3.4.7 Reproducibility

The VMM binary, the kernel image, the rootfs, the NVIDIA driver, and the CUDA runtime are versioned. The configuration JSON is checked into the artifact repository along with a shell harness that binds the GPU to `vfio-pci`, runs a batch of trials, and restores the host driver. The guest init script and the probe program are included in the rootfs and in source form. A README lists required kernel parameters for IOMMU and notes the expected sysfs path for the GPU. With these artifacts, another operator with similar hardware can rerun the three measurement suites and compare to the baselines.

# 4 Results

We report four measurements aligned with the evaluation goals: (i) cold-start latency until the GPU is usable, (ii) guest footprints when idle vs. under load, (iii) an end-to-end VectorAdd microbenchmark, and (iv) host memory headroom in the presence of a VFIO-attached guest. Cold-start uses 20 runs; medians are shown where applicable.

## 4.1 Cold-start to GPU usability

Cold-start is measured from VMM boot entry to the guest emitting `GPU_READY` after the NVIDIA driver initializes, a CUDA context is created, a fixed device allocation succeeds, and a trivial kernel completes. Figure 5 shows the medians over 20 runs: $\sim$1.58 s for a normal microVM and $\sim$5.14 s with GPU passthrough. The longer path with passthrough includes PCI/ACPI enumeration, VFIO/IOMMU setup, and the first CUDA context bring-up.

## 4.2 Guest footprints: idle vs. CUDA-active

We profile three short windows in the guest: idle with no GPU, idle with the GPU attached and driver loaded, and a CUDA-active window running the VectorAdd harness. Table 1 reports medians for each

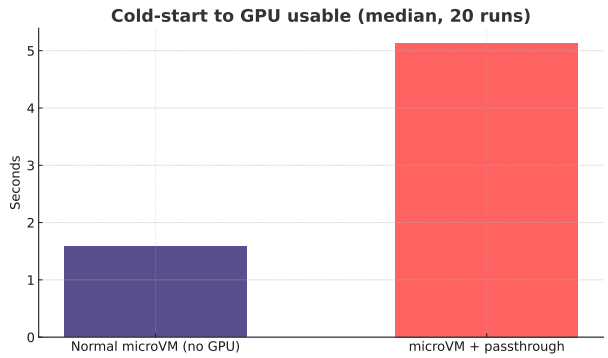


**Figure 5:** Cold-start to GPU-usable (median across 20 runs).

window. Both idle cases show 0.000 s user, 0.000 s system, and RSS = 1.8 MiB. During the CUDA-active window, the process accounts for 30.090 s user time, 1.970 s system time, and RSS = 103.6 MiB. In other words, attaching the device does not change idle CPU or memory in this setup; activity is reflected directly in the harness' CPU time and a modest working set.

**Table 1:** Guest process footprints per window (medians).

| State | User [s] | Sys [s] | Max RSS [MiB] |
|---|---|---|---|
| Idle (no GPU) | 0.000 | 0.000 | 1.8 |
| Idle (GPU attached) | 0.000 | 0.000 | 1.8 |
| CUDA-active (VectorAdd) | 30.090 | 1.970 | 103.6 |

## 4.3 VectorAdd end-to-end latency

VectorAdd is ran with $N = 2^{20}$ and 25 repetitions. Figure 6 shows the median total wall time: 189.911 ms on the host and 674.875 ms inside the microVM with passthrough. The phase medians in Tables 2 and 3 localize where time is spent. Context creation is 113.025 ms (host) vs. 582.257 ms (microVM). These measurements separate first-use effects (notably context creation) from the steady compute path.

**Table 2:** VectorAdd medians - init phases (milliseconds).

| | Ctx | Module | 3×Alloc |
|---|---|---|---|
| Host (bare metal) | 113.025 | 1.552 | 0.200 |
| microVM + passthrough | 582.257 | 0.174 | 0.749 |

**Table 3:** VectorAdd medians - transfers & launch (milliseconds).

| | H2D(A+B) | Host r/t | D2H(C) |
|---|---|---|---|
| Host (bare metal) | 0.372 | 0.028 | 0.208 |
| microVM + passthrough | 0.589 | 0.079 | 0.298 |

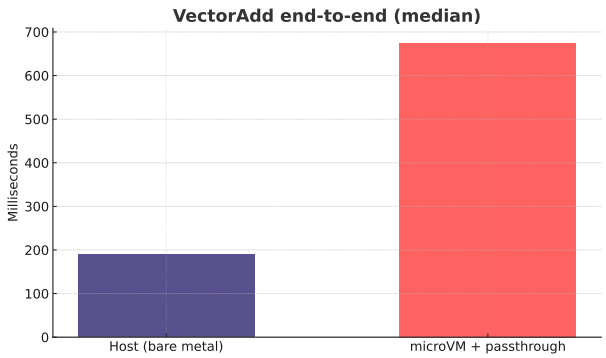


**Figure 6:** VectorAdd total wall time (median).

**Table 4:** VectorAdd total wall time (median).

| Mode | Total [ms] |
|---|---|
| Host (bare metal) | 189.911 |
| microVM + passthrough | 674.875 |

## 4.4 Host memory headroom with a VFIO GPU guest

Figure 7 summarizes `MemAvailable` on the host across three steps. With all VMs down the host reports 380,765,368 kB. After the GPU-passthrough microVM boots, `MemAvailable` is 346,313,460 kB. After adding eight more idle microVMs, it is 336,584,136 kB. While the additional idle guests change headroom only slightly, creation of a microVM with GPU-passthrough dominates in this configuration.

# 5 Discussion

The results point to a simple trade-off. Adding an opt-in PCI/VFIO path preserves Firecracker's minimal default for non-GPU guests. However, it shifts a noticeable delay into the cold path and reduces host memory headroom because of pinned DMA mappings. For the small VectorAdd we measured, the steady compute path is close to host behavior; what users experience in scale-to-zero settings is driven by lifecycle costs around boot and first CUDA use rather than steady execution.

Cold start is the largest cost we see. Figure 5 shows medians of ∼1.58 s for a normal microVM and ∼5.14 s with GPU passthrough. The phase breakdown in Tables 2 and 3 explains this gap: context creation grows from 113.025 ms on bare metal to 582.257 ms in the microVM, while module load, buffer allocations,

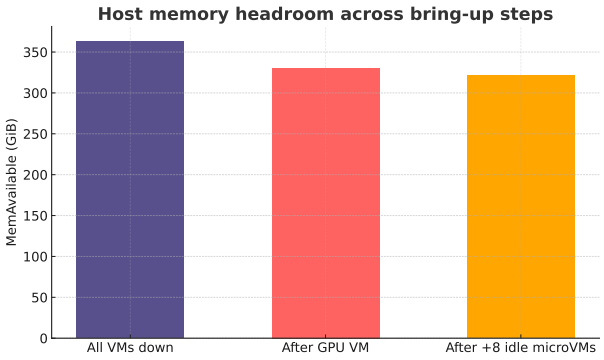


**Figure 7:** `MemAvailable` steps on the host: all VMs down, after GPU VM boot, after +8 idle microVMs.

transfers, and the host round-trip stay small. This points to PCI/ACPI enumeration, VFIO/IOMMU setup, and first CUDA context bring-up as the main sources of extra time, not a recurring penalty once the context exists.

Once warm, the steady path is comparatively flat. Figure 6 shows total medians of 189.911 ms (host) versus 674.875 ms (microVM). Most of this difference comes from first-use work; the repeating pieces remain sub-millisecond on both platforms (H2D(A+B) 0.372 → 0.589 ms, D2H(C) 0.208 → 0.298 ms, host round-trip 0.028 → 0.079 ms). Our benchmark is intentionally small to expose first-use effects, so we do not claim results for larger models or long-running throughput, but the direction is clear: the VMM path does not dominate the steady compute phase for this case.

The guest's own idle footprint is stable. Table 1 shows 0.000 s user and system time and RSS = 1.8 MiB for both idle windows, with activity showing up only during the CUDA run (30.090 s user, 1.970 s system, 103.6 MiB RSS). The host view is different once a VFIO device is attached. Figure 7 shows a large drop in `MemAvailable` at the moment the GPU microVM comes up, consistent with pinned pages for DMA, while adding eight more idle microVMs without GPUs changes headroom only slightly. In other words, the first GPU guest tightens consolidation even when it is idle.

From an operator's perspective, the upside of this design is simplicity and isolation: PCI is off by default, drivers are stock, BAR programming is conventional, interrupts use MSI/MSI-X, and the IOMMU fences DMA to mapped guest regions. The downside is that scale-from-zero gets slower by seconds, and pinned memory eats into headroom. Some of this can be hidden with warm pools or context persistence (which costs residency), or reduced with tighter DMA windows and smaller guest RAM (which costs coordination). A separate practical limit is that full passthrough dedicates a device to a single microVM; this caps consolidation compared to sharing schemes and makes naive serverless use harder.

Finally, our setup does not include hotplug, migration, or snapshot/restore for assigned GPUs, so a platform built on this path either accepts a longer cold start or keeps instances around. That is acceptable for long-lived inference services, but cuts against the grain of bursty serverless workloads that want to scale to zero quickly.

# 6 Conclusion and Future Work

This work shows that a Firecracker microVM can operate a GPU through an opt-in PCI/VFIO path under IOMMU protection while running unmodified vendor drivers, and it quantifies the cost of that choice. In our measurements, the dominant penalty is on the cold path: time from VM creation to a GPU-ready state grows from a normal microVM median of ∼1.58 s to ∼5.14 s with passthrough, and first CUDA context creation expands from 113.025 ms on the host to 582.257 ms in the guest. The steady compute path for a small kernel remains close to host in absolute terms, with per-phase times in the sub-millisecond range. On the host, memory headroom drops sharply when the GPU guest comes up, consistent with pinned DMA mappings; adding several more idle non-GPU microVMs has only a small additional effect. In short, the mechanism is clean and minimal, but the lifecycle costs make naive scale-to-zero unattractive and reduce consolidation headroom even when GPU guests are idle.

Future work might move from whole-device passthrough to assignable virtual GPUs (vGPU) that still look like PCIe devices to the guest and can therefore be bound by the same VFIO path. Two concrete paths are worth evaluating: (i) vendor virtual GPUs (SR-IOV or mediated devices) that enumerate as PCIe functions and can be assigned to different microVMs, and (ii) NVIDIA MIG partitions carved up to the device's MIG limit and exposed as independently schedulable slices.

Our device model is already compatible with this

mode-enumeration, BAR mapping, and MSI delivery are unchanged-so the key questions become whether boot-to-ready improves with pre-created vGPU/MIG instances, how much cross-tenant interference shows up under load, how far context persistence and warm pools can hide first-use costs without burning too much residency, and whether tighter DMA windowing and smaller guests recover enough headroom to make serverless-style scaling viable. A follow-up should answer these by repeating the cold-start and steady-state measurements under vGPU/MIG assignment and including larger inference kernels that better represent production workloads.

# References


[1] Vfio device assignment (common diagram). `https://wiki.qemu.org/images/c/cc/Vfio-device-assignment-common.png`, 2017. Accessed 2025.

[2] Firecracker architecture diagram. `https://firecracker-microvm.github.io/img/diagram-desktop@3x.png`, 2019. Accessed 2025.

[3] Alexandru Agache, Marc Brooker, et al. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 419–434, 2020.

[4] AWS. *Firecracker MicroVM: Design & Performance*, 2019.

[5] Paulo Castro, Vatche Ishakian, Vinod Muthusamy, and Aleksander Slominski. The rise of serverless computing. *Communications of the ACM*, 62(12):44–54, 2019.

[6] Cheol-Ho Hong, Iain Spence, and Dimitrios Nikolopoulos. Gpu virtualization and scheduling methods: A comprehensive survey. *ACM Computing Surveys*, 50(3):1–37, 2017.

[7] Jian Huang, Wei Lin, Wentai Wu, et al. On efficiency, fairness and security in ai accelerator resource sharing: A survey. *ACM Computing Surveys*, 57(9), 2025.

[8] The Linux Kernel Documentation. *VFIO - Virtual Function I/O*, 2023. Accessed 2025.

[9] Alexandra Sasha Luccioni, Yacine Jernite, and Emma Strubell. Power hungry processing: Watts driving the cost of ai deployment? In *ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pages 240–253, 2024.

[10] Firecracker Maintainers. Gpu and pcie support in firecracker (issue #4845). GitHub Discussion, 2020.

[11] Matteo Manco, Antti Kantee, Sebastian Ruocco, and Ian Pratt. My vm is lighter (and safer) than your container. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP)*, pages 218–233, 2017.

[12] NVIDIA. *Virtual GPU (vGPU) Software User Guide v12.0*, 2023.

[13] Red Hat Customer Portal. *Managing GPU Devices (RHEL 8 Virtualization)*, 2023.

[14] Jennifer Schleier-Smith, Vibhav Sreekanti, Aditya Khandelwal, et al. What serverless computing is and should become: The next phase of cloud computing. *Communications of the ACM*, 64(5):76–84, 2021.

[15] Mohammad Shahrad et al. Characterizing and optimizing the serverless workload at a large cloud provider. In *USENIX Annual Technical Conference (ATC)*, pages 205–218, 2020.

[16] Ryan Shea and Jiangchuan Liu. On gpu passthrough performance for cloud gaming. In *IEEE/ACM NetGames*, 2013.

[17] Cristina Silvano, Daniele Ielmini, Fabrizio Ferrandi, et al. Deep learning hardware accelerators for heterogeneous hpc platforms: A survey. *ACM Computing Surveys*, 58(9), 2025.

[18] Ben Yassour, Muli Ben-Yehuda, and Oren Wasserman. On the dma mapping problem in direct device assignment. In *ACM SYSTOR*, 2010.